



Проложите ребенку путь в ИТ



2K+

Охват за 30 дней

билайн

4,19 Оценка работодателя

60,12 Рейтинг

360 Подписчики

Подписаться

**kino6052**

31 авг 2023 в 16:01

Почему код становится legacy?



7 мин



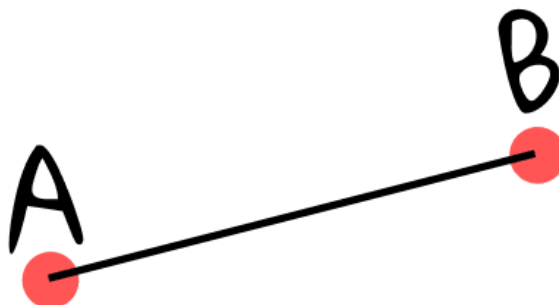
14K

Блог компании билайн, Программирование*, Качество кода*, Дизайн

Мнение

Небольшая метафора

Написание кода похоже на соединение двух точек. Ожидаемо, что самым простым путем будет нарисовать прямую линию между точками А и Б.



Для большинства простых случаев это работает, но вот в реальной жизни, кроме соединения точек, вам придется еще и обходить различные препятствия. Но в целом — тут тоже никакой

РЕКЛАМА

**Вложение в навыки**

Лучшая стратегия в кризис



Теперь давайте заставим эти препятствия двигаться. Медленно, но достаточно для того, чтобы

вызвать проблемы с необходимостью переподключения точек. Это уже не такая простая прямая линия, и начинает выглядеть куда серьезнее.

А если мы заставим двигаться не только препятствия, но и сами точки? Вдобавок убедимся, что эти точки не приклеены к линиям, и вам придется следить за ними, чтобы они оставались соединенными. Начинает немного бесить?

Но и это еще не всё. Представьте, что мы знаем лишь приблизительное положение точек, и единственный способ узнать его — запрашивать их местоположение каждые 5 минут? Походит на безумие, или еще нет?



Усложним еще немного. Допустим, мы знаем **только приблизительное положение** точек.

Вот это уже в значительной степени похоже на то, как происходит разработка в реальном мире.

Обсуждение

В контексте этой метафоры точки — это функциональные требования к тому, что должно делать программное обеспечение. Замечательно, когда мы точно знаем, правильно ли наш код соединяет их. Однако на практике эти требования меняются (отсюда и движение точек), и нам приходится соответствующим образом обновлять наш код. Тем не менее, мы не всегда можем легко определить, правильно ли мы соединили точки — для этого надо проводить тщательное тестирование.

Препятствия могут быть ограничениями в технологии, производительности, программировании, различными процессами и всем остальным, что может встать на нашем пути. Всегда есть что-то, с чем мы как решатели проблем уже привыкли сталкиваться.

Само собой, при такой динамической ситуации код, который изначально был написан с наилучшими намерениями и практиками, превращается в уродливую кривую, свидетельствующую о мучениях, через которые пришлось пройти разработчику, чтобы заставить его работать.

Такой код известен как **legacy** — код, который очень сложно изменить без полного краха и который обходится так же дорого, как и полная перепись с нуля.

Да, с классической точки зрения, **legacy** — это старый код, который нужно поддерживать, как интернет

РЕКЛАМА



Вложение в навыки

Лучшая стратегия в кризис



Но в современном обиходе этот термин более широкий и обозначает плохой ломкий код.

Как запутанная пряжа, устаревший код обычно проявляет высокую связность между логическими

блоками, которую так же сложно размотать.

Отсюда возникает вопрос, можно ли что-то сделать, чтобы избежать устаревания нашего кода.

Как решить эту проблему?

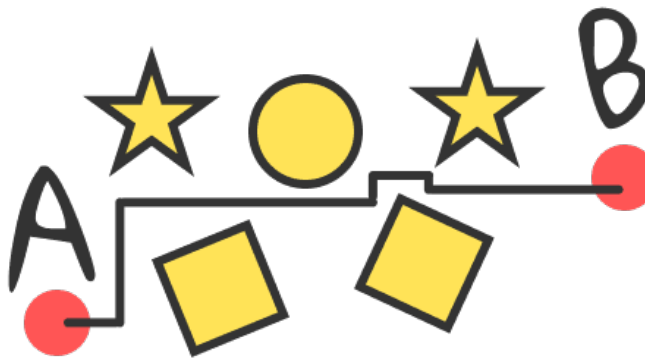
Частично она решается с помощью так называемых шаблонов проектирования - точечных решений, которые мы можем собрать вместе, подобно конструктору Лего, чтобы сделать процесс быстрее и более контролируемым.

Но шаблонов проектирования недостаточно. Гораздо важнее знать, что мы быстро движемся **в правильном направлении**, нежели **правильно двигаемся в неправильном**. И да, в мире полно устаревшего кода, который следует лучшим шаблонам проектирования. Фактически, чрезмерное использование шаблонов проектирования сверх необходимого называется "оверинжинерингом".

Мы должны как можно быстрее узнавать, где находятся точки из нашего примера, и для этого нам нужен автоматизированный мгновенный способ получения обратной связи. В этом помогает тестирование вашего кода. Да, не каждый тест подходит, но [тестирование по стратегии черного ящика](#) специально проверяет именно точки, а не какие-то части по пути.

В общем, чтобы решить проблему постоянно меняющегося положения точек и препятствий, нам нужно выполнить два условия:

1. Как можно быстрее узнавать, что мы быстро движемся в правильном направлении (обратная связь о правильности в виде черного ящика тестов)
2. Использовать заранее продуманные решения для быстрого и контролируемого построения пути (шаблоны проектирования).



Что в итоге

Эти два пункта выше очень просто звучат, но их не так-то просто достигнуть в контексте разработки пользовательского интерфейса, чем мы и попытаемся заняться в этом цикле статей.

Однако основное внимание будет уделено первому аспекту (немедленной обратной связи), так как именно он часто оказывается наиболее болезненным, в то время как дизайн-паттерны

РЕКЛАМА



Вложение в навыки

Лучшая стратегия в кризис



Почему разработка пользовательского интерфейса особенно склонна к Legacy?

Используя метафору с линией из примера выше, мы пришли к определению того, что такое legacy-код:

Legacy-код — это код, который очень сложно изменить, ничего не сломав в процессе, и его переписывание обходится так же дорого, как написание с нуля.

Такой код появляется, когда у нас есть сложные и постоянно меняющиеся ограничения и требования, обратная связь о корректности задерживается, а объем кодовой базы — постоянно растет.

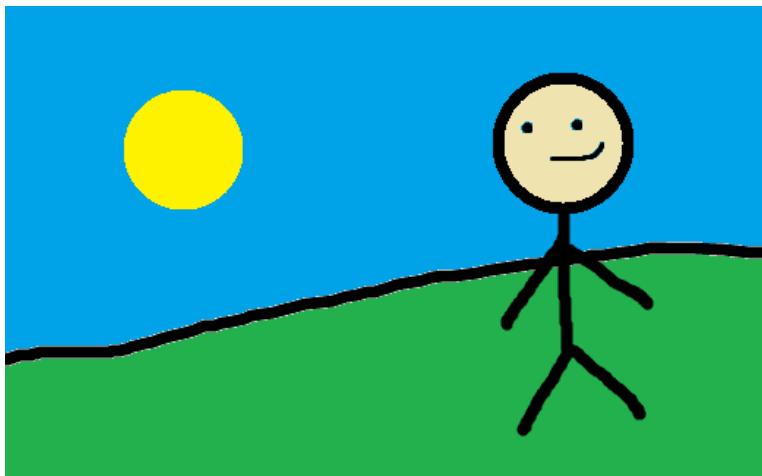
И это очень похоже на разработку пользовательского интерфейса в общем понимании этого процесса. И веб-разработка пользовательского интерфейса не является исключением. Больше скажу — возможно, нет другой области, где требования меняются так же часто, как в областях, связанных с пользовательским интерфейсом.

Дисклеймер: Все приведенные ниже примеры являются вымышленными, упрощенными и нужны только для иллюстрации — любое сходство с реальными людьми случайно.

От идеи до legacy

Greenfield — проект одного человека, создаваемый с нуля

Есть такие счастливые люди, которые начинают работать над проектом вообще с нуля. Пожалуй, это самый крутой этап для разработчика.



Когда вы получаете требования, идея того, что необходимо создать, обычно ясна. Список задач свежий и хорошо организован, все понятно, а дизайн по большей части почти готов.

Итак, вы начинаете работать над проектом, настраиваете конфигурации и прочее. Но как только этот этап остается позади, игра начинается — теперь вы создаете.

Пока вы создаете свои первые компоненты, вам четко заметен быстрый прогресс. Ещё бы — вам не нужны тесты, ведь вы можете перемещаться по всему приложению всего в пару кликов. Свой

РЕКЛАМА



Вложение в навыки

Лучшая стратегия в кризис



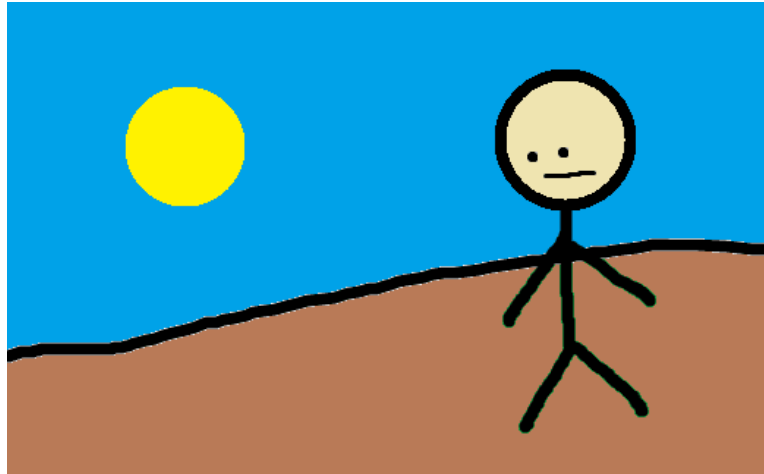
С некоторыми неудачами (мы же с вами в реальном мире живем) вы справляетесь. Всё сделано

вовремя, все довольны.

Конец.

Проект Brownfield

Более вероятен вариант, при котором вы окажетесь в другой ситуации и будете работать над чьим-то проектом «с нуля».



Вам будет гораздо сложнее понять мыслительный процесс человека, который создавал приложение до вас. Как он думал, что делал, зачем. Скорее всего, он не обращал много внимания на что-то, кроме требований, поэтому тестирования тут будет мало (если оно вообще будет). А логика будет куда более запутанной из-за упрощения разработки.

Теперь, получив требования, вы сможете только догадываться, сколько времени потратите на реализацию.

Пока вы начинаете работать над проектом, вы в основном отлаживаетесь через код. Вы больше не можете кликать по пользовательскому интерфейсу и знать, где находится каждый элемент. По мере разработки вы стараетесь как можно меньше изменять исходную логику. Пошагово вы тесно связываете одну функцию с другой, пытаетесь понять исходный код и закончить работу к концу спринта.

Может быть, у вас и не будет возможности надлежащим образом протестировать что-то, так как вы уже будете по уши загружены требованиями и отладкой.

Тем не менее, если вам повезет, вы начнете писать тесты для своего кода. Наверняка это будет уже история про тестирование [белого ящика](#) (структурное), то есть ваши тесты будут привязаны к реализации, а не станут тестировать общую логику с входными и выходными данными. А это станет проблемой.

Шаг за шагом, и кодовая база станет тем, что называется legacy.

Legacy-проект

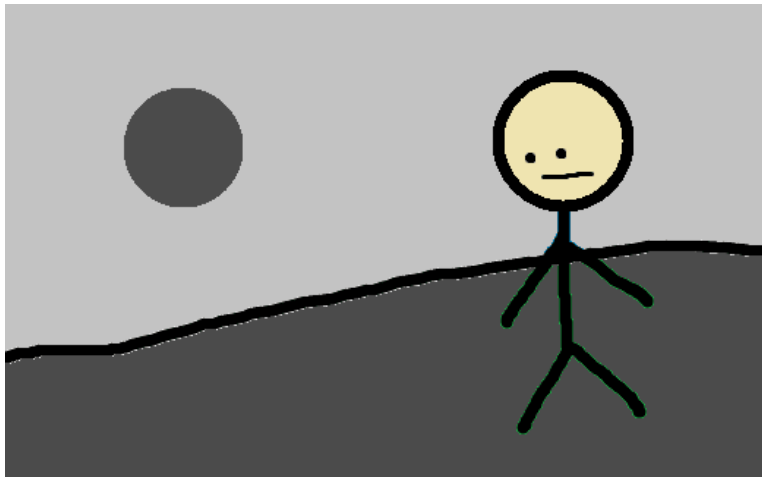
РЕКЛАМА



Вложение в навыки

Лучшая стратегия в кризис





Здесь почти нет надежды на рефакторинг устаревшей части. Цикл обратной связи для большинства приложений будет ощутимо задерживаться, а процесс покрытия приложения тестами всегда будет отставать и подрывать мотивацию.

Заключение

Разработка пользовательского интерфейса подвержена устареванию, потому что быстрая обратная связь и эффективные шаблоны проектирования, позволяющие обеспечить модульность, не включаются в проект на этапе его создания с нуля.

Далее в цикле:

- Так как же сделать пользовательский интерфейс легкотестируемым и легкоизменяемым?
- Decoupler MVU или как реализовать MVU архитектуру для UI приложений

Полезные ссылки

1. [Black Box Testing: An In-Depth Tutorial](#) — Этот tutorial на Guru99 предоставляет всестороннее руководство по тестированию черного ящика. В нем рассматриваются его методы, типы, приемы, преимущества и недостатки.
2. [Design patterns](#) — Подробное руководство по шаблонам проектирования программного обеспечения. Каждый шаблон подробно объясняется с примерами, чтобы лучше понять, когда и где их использовать.
3. [SOLID Principles: Explanation and examples](#) — Этот пост на FreeCodeCamp разбирает принципы SOLID в понятной форме с большим количеством примеров.
4. [Understanding Legacy Code](#): Этот сайт — отличный источник информации для понимания legacy-кода и стратегий эффективной работы с ним. Поможет вам ориентироваться и улучшать legacy-системы.
5. [Greenfield vs Brownfield](#): В этой статье на LinkedIn Giorgi Bastos подробно рассматривает

РЕКЛАМА

**Вложение в навыки**

Лучшая стратегия в кризис



6. [Greenfield vs Brownfield in Software Development](#): Synorpeck предоставляет подробное сравнение между разработкой программного обеспечения greenfield и brownfield, дополнительно улучшая понимание этих концепций и их влияния на разработку пользовательского интерфейса.

7. [Advantages and Disadvantages of White Box Testing](#): На JavaTpoint опубликовано сбалансированное представление о тестировании белого ящика и подробное обсуждение его преимуществ и ограничений.
8. [White Box Testing: Pros and Cons](#): Segue Technologies предлагает другую точку зрения на тестирование белого ящика, подробно описывая его плюсы и минусы. Эта статья поможет вам понять, почему тестирование белого ящика может быть частью проблемы при поддержке кода пользовательского интерфейса.

Теги: философия программирования, программирование, легаси, legacy, поддержка кода, UI

Хабы: Блог компании билайн, Программирование, Качество кода, Дизайн



+20



64



16

Редакторский дайджест

Присылаем лучшие статьи раз в месяц

Подписаться

Оставляя почту, я принимаю [Политику конфиденциальности](#) и даю согласие на получение рассылок



2K+

Охват за 30 дней

билайн

[Сайт](#) [Сайт](#)



23

Карма

Кирилл Новик @kino6052

Striving to make scalable UI development possible

Подписаться



Хабр доступен 24/7 благодаря поддержке друзей

РЕКЛАМА



Вложение в навыки

Лучшая стратегия в кризис



Хабр Карьера

КУРСЫ

для профессионального
роста и повышения
зарплаты



Хабр Курсы для всехРЕКЛАМА

Практикум, Хекслет, SkyPro, авторские курсы — собрали всех и попросили скидки. Осталось выбрать!

[Перейти](#)

[Комментарии 16](#)

Публикации

[ЛУЧШИЕ ЗА СУТКИ](#) [ПОХОЖИЕ](#)

-  **ru_vds**
6 часов назад

Установка OpenClaw на VPS

 **Средний**  8 мин  5.6K

[Обзор](#)

 **+23**  20  2
-  **PatientZero**
6 часов назад

Если if вас замедляют, откажитесь от них

 **Простой**  4 мин  8.7K

[Перевод](#)

 **+22**  22  36

РЕКЛАМА

**Вложение в навыки**
Лучшая стратегия в кризис**Как Яндекс победил Google? Или почему блокировки — худший способ
помочь отечественным технологиям**

- Простой**

9 мин

22K

Ретроспектива

Из песочницы

Recovery Mode

+19

20

149
- cnet**
 8 часов назад

Про двуного-колёсных роботов

9 мин

7.2K

+17

14

13
- Andrew42**
 3 часа назад

Вы пустили ИИ-агента в репозиторий, теперь разбираемся, что он может сломать

Средний

22 мин

5.6K

Кейс

+16

7

2
- dem1dov1van**
 7 часов назад

Как я создавал платформу для внутренних торгов мебелью и почему для этого выбрал PocketBase + Nuxt 3

Простой

6 мин

6.6K

+16

4

0
- SimpleOne_it**
 6 часов назад

Ваша кодовая база умрёт через 7 лет. Считаем на пальцах

Средний

9 мин

5.7K

Аналитика

+15

14

9
- daniilbelikov**
 3 часа назад

Сколько я заработал в RuStore за 6 месяцев

Простой

5 мин

9.1K

Мнение

+14

21

17

РЕКЛАМА



Вложение в навыки
 Лучшая стратегия в кризис



Опенсорсим ux_navigation — декларативную навигацию для Flutter

МИНУТОЧКУ ВНИМАНИЯ

Как сегодня проверяют навыки работы с ИИ на собеседованиях

Записи Хабр Семинара про смысл работы

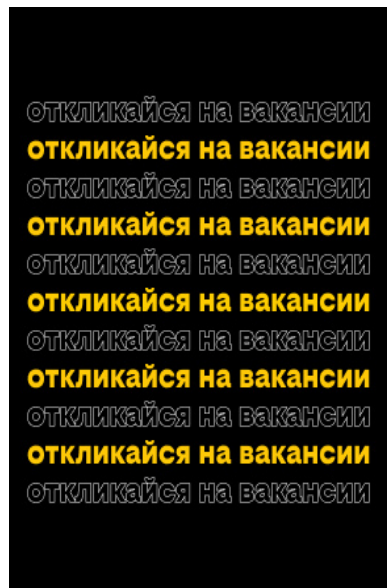
Экскурсия по Хабру для маркетологов



ИНФОРМАЦИЯ

Сайт	beeline.ru
Дата регистрации	27 февраля 2023
Дата основания	2 июня 1992
Численность	свыше 10 000 человек
Местоположение	Россия

ВИДЖЕТ



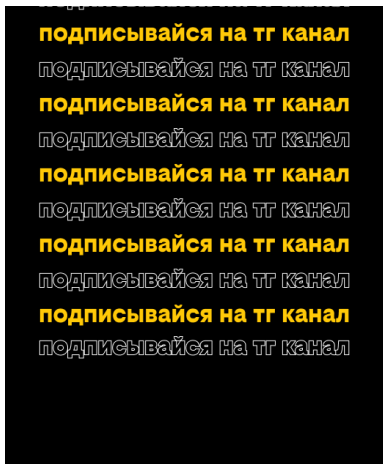
РЕКЛАМА



Вложение в навыки
Лучшая стратегия в кризис







БЛОГ НА ХАБРЕ

30 дек 2025 в 18:57

Как мы отключали 3G

12K 50

28 окт 2025 в 13:43

Как мы вырастили ML-фреймворк внутри компании: эволюция, ошибки и инсайты

6.2K 0

21 окт 2025 в 14:47

Делаем кастомное параллельное чтение по JDBC в Spark 3.0.1

5.7K 2

19 авг 2025 в 13:41

Работа на удалёнке: как начать нахаживать норму шагов

15K 27

12 авг 2025 в 13:11

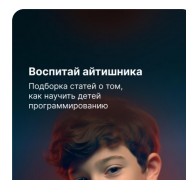
Автоматизированная оценка стабильности скоринговых моделей на основе временных рядов метрик

6.1K 0

ИСТОРИИ



РЕКЛАМА



Вложение в навыки
Лучшая стратегия в кризис



Годнота из блогов компаний



Большие данные, алгоритмы и



Захлестнуло радиоволной



Схватил за мозг



Как учить детей

другие секреты ретейла

программированию

Настройка языка

Техническая поддержка

© 2006–2026, Habr

РЕКЛАМА



Вложение в навыки
Лучшая стратегия в кризис

